USER SPACE, KERNEL, HARDWARE

EXPLORING LINUX DRIVERS WITH RUST



Cyril Marin & René Ribaud



Un Grand MERCI à nos sponsors 2026





CGI

open

KAIZEN SOLUTIONS

aws

CRITEO

Moody's

clever cloud



AVISTO

KLS GROUP

kelkoogroup

VISEO
POSITIVE DIGITAL MAKERS

alma

sopra steria

PeriScop

enalean
software engineering is now

HACK YOUR JOB

PingID

zenika

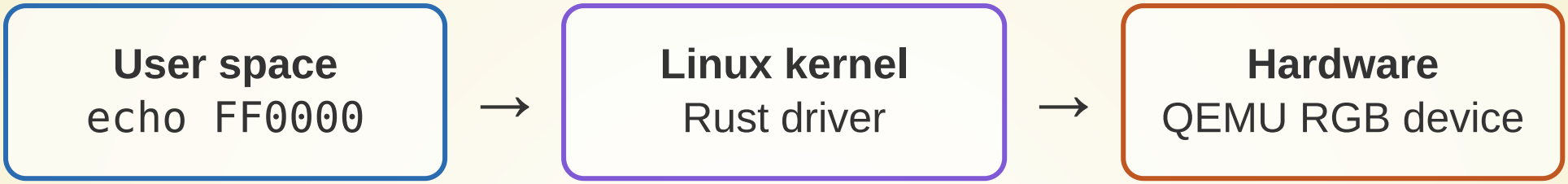
WHO ARE WE?





Cyril Marin
Cloud engineer

ONE COMMAND, THREE WORLDS



We will build every boundary ourselves.

OUR ROUTE

1. Build a virtual RGB peripheral in QEMU
2. Register a Rust character device
3. Talk to hardware through hard-coded MMIO
4. Discover resources through the Device Tree
5. Add an ioctl and move work off the caller
6. Target hardware fade completion through an IRQ

Explain → inspect code → demonstrate

1 – FIRST, BUILD THE HARDWARE

QEMU as our virtual electronics bench

WE ARE ACTING LIKE A BOARD DESIGNER

A HARDWARE DESIGNER DECIDES

- register semantics
- address ranges
- interrupt wiring
- firmware description

AT OUR SCALE

- C structs hold device state
- QEMU callbacks emulate bus accesses
- the `virt` board maps the device
- QEMU emits a Device Tree node

A TINY MMIO CONTRACT

Offset	Register	Meaning
`0x00`	RED	8-bit red channel
`0x04`	GREEN	8-bit green channel
`0x08`	BLUE	8-bit blue channel
`0x0c`	APPLY	write 1 to apply
`0x10`	STATUS	busy / ready / error

Physical base: `0x090d0000` · window: `0x100`

QEMU: READS AND WRITES BECOME CALLBACKS

```
1  static const MemoryRegionOps rgb_led_ops = {
2      .read = rgb_led_read,
3      .write = rgb_led_write,
4      .endianness = DEVICE_NATIVE_ENDIAN,
5  };
6
7  static void rgb_led_write(void *opaque, hwaddr offset,
8                            uint64_t value, unsigned size)
9  {
10     RGBLedState *s = opaque;
11     switch (offset) {
12     case RGB_LED_REG_RED:   s->red = value & 0xff; break;
13     case RGB_LED_REG_GREEN: s->green = value & 0xff; break;
14     case RGB_LED_REG_BLUE:  s->blue = value & 0xff; break;
15     case RGB_LED_REG_APPLY: if (value == 1) rgb_led_apply(s); break;
16     }
17 }
```

QEMU: READS AND WRITES BECOME CALLBACKS

```
static const MemoryRegionOps rgb_led_ops = {
    .read = rgb_led_read,
    .write = rgb_led_write,
    .endianness = DEVICE_NATIVE_ENDIAN,
};

static void rgb_led_write(void *opaque, hwaddr offset,
                          uint64_t value, unsigned size)
{
    RGBLedState *s = opaque;
    switch (offset) {
    case RGB_LED_REG_RED:   s->red = value & 0xff; break;
    case RGB_LED_REG_GREEN: s->green = value & 0xff; break;
    case RGB_LED_REG_BLUE:  s->blue = value & 0xff; break;
    case RGB_LED_REG_APPLY: if (value == 1) rgb_led_apply(s); break;
    }
}
```

QEMU: PLACE IT ON THE `virt` BOARD

```
[VIRT_RGB_LED] = { 0x090d0000, 0x00000100 },

sysbus_create_simple("rgb-led", base, NULL);

nodename = g_strdup_printf("/rgb-led@%" PRIx64, base);
qemu_fdt_add_subnode(ms->fdt, nodename);
qemu_fdt_setprop_string(ms->fdt, nodename,
                        "compatible", "qemu,rgb-led");
qemu_fdt_setprop_sized_cells(ms->fdt, nodename, "reg",
                             2, base, 2, size);
```

The address map and firmware description are two views of the same contract.

QEMU: PLACE IT ON THE `virt` BOARD

```
[VIRT_RGB_LED] = { 0x090d0000, 0x00000100 },

sysbus_create_simple("rgb-led", base, NULL);

nodename = g_strdup_printf("/rgb-led@%" PRIx64, base);
qemu_fdt_add_subnode(ms->fdt, nodename);
qemu_fdt_setprop_string(ms->fdt, nodename,
                        "compatible", "qemu,rgb-led");
qemu_fdt_setprop_sized_cells(ms->fdt, nodename, "reg",
                             2, base, 2, size);
```

The address map and firmware description are two views of the same contract.

DEMO 1 – NO DRIVER, JUST REGISTERS

1. Boot the prepared VM with `make run`
2. `Ctrl-a c`: enter the QEMU monitor
3. Inspect physical memory, then return with `Ctrl-a c`
4. Use `devmem` for RED, GREEN, BLUE and APPLY
5. Observe `rgb-led: apply RGB(255, 0, 0)`

```
devmem 0x090d0000 32 0xff
devmem 0x090d0004 32 0x00
devmem 0x090d0008 32 0x00
devmem 0x090d000c 32 0x01
```

2 — A CHARACTER DEVICE, BUT NO HARDWARE

Build the user space ↔ kernel boundary first

WHAT HAPPENS TO `echo FF0000 > /dev/rgb`?



`/dev/rgb` is a rendezvous point, not the hardware itself.

MAJOR, MINOR, AND mknod

Major

selects the registered driver

Minor

selects one device managed by it

```
cat /proc/devices
# 240 rgb

mknod /dev/rgb c 240 0
ls -l /dev/rgb
# crw-r--r-- ... 240, 0
```

REGISTERING THE RUST CHARACTER DEVICE

```
let mut raw_fops: bindings::file_operations =
    unsafe { MaybeUninit::zeroed().assume_init() };
raw_fops.owner = module.as_ptr();
raw_fops.open = Some(rgb_open);
raw_fops.write = Some(rgb_write);

to_result(unsafe {
    bindings::alloc_chrdev_region(
        &mut dev, 0, 1, c_str!("rgb").as_char_ptr())
})?;

bindings::cdev_init(cdev_ptr, ptr::from_ref(&*fops));
to_result(unsafe { bindings::cdev_add(cdev_ptr, dev, 1) })?;

let major = (dev >> bindings::MINORBITS) as u32;
```

Stable allocations and explicit cleanup bridge Rust ownership with C kernel APIs.

REGISTERING THE RUST CHARACTER DEVICE

```
let mut raw_fops: bindings::file_operations =
    unsafe { MaybeUninit::zeroed().assume_init() };
raw_fops.owner = module.as_ptr();
raw_fops.open = Some(rgb_open);
raw_fops.write = Some(rgb_write);

to_result(unsafe {
    bindings::alloc_chrdev_region(
        &mut dev, 0, 1, c_str!("rgb").as_char_ptr())
})?;

bindings::cdev_init(cdev_ptr, ptr::from_ref(&*fops));
to_result(unsafe { bindings::cdev_add(cdev_ptr, dev, 1) })?;

let major = (dev >> bindings::MINORBITS) as u32;
```

Stable allocations and explicit cleanup bridge Rust ownership with C kernel APIs.

REGISTERING THE RUST CHARACTER DEVICE

```
let mut raw_fops: bindings::file_operations =
    unsafe { MaybeUninit::zeroed().assume_init() };
raw_fops.owner = module.as_ptr();
raw_fops.open = Some(rgb_open);
raw_fops.write = Some(rgb_write);

to_result(unsafe {
    bindings::alloc_chrdev_region(
        &mut dev, 0, 1, c_str!("rgb").as_char_ptr())
})?;

bindings::cdev_init(cdev_ptr, ptr::from_ref(&*fops));
to_result(unsafe { bindings::cdev_add(cdev_ptr, dev, 1) })?;

let major = (dev >> bindings::MINORBITS) as u32;
```

Stable allocations and explicit cleanup bridge Rust ownership with C kernel APIs.

THE WRITE CALLBACK CROSSES A TRUST BOUNDARY

```
let user_ptr = UserPtr::from_ptr(user_buffer.cast_mut().cast());
let mut reader = UserSlice::new(user_ptr, count).reader();
reader.read_slice(&mut buffer[..count])?;

let digits = input.strip_prefix(b"#").unwrap_or(input);
if digits.len() != 6 {
    return None;
}
Some((
    parse_byte(&digits[0..2])?,
    parse_byte(&digits[2..4])?,
    parse_byte(&digits[4..6])?,
))
```

THE WRITE CALLBACK CROSSES A TRUST BOUNDARY

```
let user_ptr = UserPtr::from_ptr(user_buffer.cast_mut().cast());
let mut reader = UserSlice::new(user_ptr, count).reader();
reader.read_slice(&mut buffer[..count])?;

let digits = input.strip_prefix(b"#").unwrap_or(input);
if digits.len() != 6 {
    return None;
}
Some((
    parse_byte(&digits[0..2])?,
    parse_byte(&digits[2..4])?,
    parse_byte(&digits[4..6])?,
))
```

DEMO 2 – LOAD, USE, UNLOAD

```
make image RGB_DRIVER_STAGE=01-character-device
scp -P 2222 .build/buildroot-output/01-character-device/\
  build/linux-6.18.7/samples/rust/rgb.ko root@localhost:/tmp/

insmod /tmp/rgb.ko
major=$(awk '$2 == "rgb" { print $1 }' /proc/devices)
mknod /dev/rgb c "$major" 0
echo FF0000 > /dev/rgb
dmesg | tail
rmmod rgb
```

3 – CROSS THE MMIO BOUNDARY

Start with a physical address baked into the driver

RESERVE → MAP → ACCESS → RELEASE

`request_mem_region`
claim ownership

→

`ioremap`
create kernel mapping

→

`writel`
perform MMIO

```
const RGB_MMIO_BASE: usize = 0x090d0000;
const RGB_MMIO_SIZE: usize = 0x1000;

let resource = bindings::request_mem_region(
    RGB_MMIO_BASE as _, RGB_MMIO_SIZE as _, c_str!("rgb").as_char_ptr());
let base = bindings::ioremap(RGB_MMIO_BASE as _, RGB_MMIO_SIZE);
```

ORDERING IS PART OF THE DEVICE PROTOCOL

```
1 bindings::writel(red as u32, self.register(RGB_REG_RED));
2 bindings::writel(green as u32, self.register(RGB_REG_GREEN));
3 bindings::writel(blue as u32, self.register(RGB_REG_BLUE));
4 bindings::writel(1, self.register(RGB_REG_APPLY));
```

volatile I/O helpers

aligned offsets

APPLY comes last

Useful for learning; unsuitable for reusable board support.

DEMO 3 – THE DRIVER REACHES QEMU

```
make image RGB_DRIVER_STAGE=02-mmio-fixed
# copy, unload the previous module, then:
insmod /tmp/rgb-mmio.ko
echo 00FF00 > /dev/rgb
```

Linux: `wrote MMIO color r=0 g=255 b=0`

QEMU: `rgb-led: apply RGB(0, 255, 0)`

4 – LET FIRMWARE DESCRIBE THE BOARD

Device Tree + platform driver

THE DEVICE TREE IS THE HAND-OFF

```
rgb-led@90d0000 {
    compatible = "qemu,rgb-led";
    reg = <0x0 0x090d0000 0x0 0x100>;
};
```

QEMU emits device node → Linux creates platform device → compatible match calls `probe`

A RUST PLATFORM DRIVER

```rust
kernel::of_device_table!(
    RGB_OF_TABLE, MODULE_OF_TABLE,
    <RgbDriver as platform::Driver>::IdInfo,
    [(of::DeviceId::new(c_str!("qemu,rgb-led")), ())]
);

impl platform::Driver for RgbDriver {
    type IdInfo = ();
    const OF_ID_TABLE: Option<of::IdTable<Self::IdInfo>> =
        Some(&RGB_OF_TABLE);

    fn probe(pdev: &platform::Device<Core>, _info: Option<&()>)
        -> Result<Pin<KBox<Self>>> {
        let hw = RgbDevice::map(pdev)?;
        // register /dev/rgb ...
    }
}
```

The driver asks for resource 0; it no longer knows the physical base.

A RUST PLATFORM DRIVER

```rust
kernel::of_device_table!(
    RGB_OF_TABLE, MODULE_OF_TABLE,
    <RgbDriver as platform::Driver>::IdInfo,
    [(of::DeviceId::new(c_str!("qemu,rgb-led")), ())]
);

impl platform::Driver for RgbDriver {
    type IdInfo = ();
    const OF_ID_TABLE: Option<of::IdTable<Self::IdInfo>> =
        Some(&RGB_OF_TABLE);

    fn probe(pdev: &platform::Device<Core>, _info: Option<&()>)
        -> Result<Pin<KBox<Self>>> {
        let hw = RgbDevice::map(pdev)?;
        // register /dev/rgb ...
    }
}
```

The driver asks for resource 0; it no longer knows the physical base.

DEMO 4 – DISCOVERY INSTEAD OF ASSUMPTIONS

```
make image RGB_DRIVER_STAGE=03-device-tree
insmod /tmp/rgb-dt.ko
dmesg | tail
cat /proc/iomem | grep -i rgb
echo 0000FF > /dev/rgb
```

Expected: `platform device matched`

5 – COMMANDS BEYOND `write`

ioctl, synchronous work, then a workqueue

A DELIBERATELY SMALL IOCTL CONTRACT

```
const RGB_IOCTL_CYCLE: u32 =
    _IO('r' as u32, 0x01);

match cmd {
    RGB_IOCTL_CYCLE =>
        hw.cycle_primary_colors(),
    _ => Err(ENOTTY),
}
```

```
RGB_IOCTL_CYCLE =
    (ord("r") << 8) | 0x01

with open("/dev/rgb", "rb+") as dev:
    fcntl.ioctl(
        dev.fileno(),
        RGB_IOCTL_CYCLE,
        0,
    )
```

STAGE 04 VS STAGE 05

04 – SYNCHRONOUS

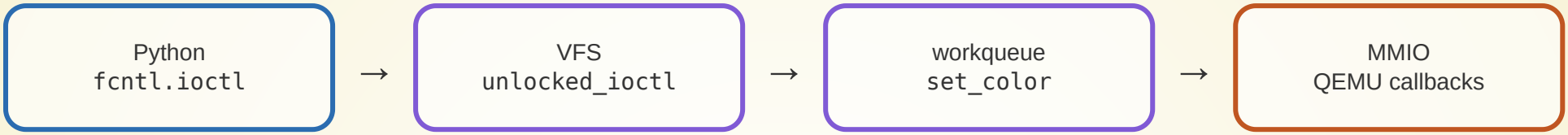
- ioctl performs the full cycle
- caller blocks for ~3 seconds
- simple lifetime model

05 – WORKQUEUE

- ioctl schedules work and returns
- kernel worker performs the cycle
- `Arc` keeps device state alive

Asynchronous software is still running the stepped color sequence.

FOLLOW THE COMMAND THROUGH ALL THREE WORLDS



One command becomes three register transactions.

DEMO 5 – IOCTL END TO END

```
make image RGB_DRIVER_STAGE=05-ioctl-async
python3 rgb_ioctl_cycle.py /dev/rgb
```

application returns

worker logs R → G → B

QEMU applies each color

STAGE 06 — PLANNED

6 – LET THE HARDWARE DO THE FADE

Completion notification through an IRQ

WHY OFFLOAD THE OPERATION?

TODAY

The kernel worker wakes up, writes each color and sleeps.

TARGET

The driver submits one command. The peripheral advances autonomously.

Submit → continue working → handle completion

THE RETURN PATH: AN INTERRUPT



The driver stops polling; the device tells us when it is done.

STAGE 06 IMPLEMENTATION CONTRACT

DRAFT — EXACT REGISTER OFFSETS STILL TO BE IMPLEMENTED

QEMU
fade command/state
virtual timer
IRQ output
migration/reset state

Rust driver
discover IRQ
request handler
submit command
ack completion
safe teardown

User space
trigger fade
receive completion
handle busy/error
observe latency

TARGET DEMO — NOT IMPLEMENTED YET

DEMO 6 – AUTONOMOUS FADE + COMPLETION IRQ

1. User program submits one fade command
2. Driver returns without performing intermediate color writes
3. QEMU executes the fade on its virtual timer
4. QEMU raises the IRQ on completion
5. Rust handler acknowledges it and notifies user space

Acceptance signal: traces show exactly one complete round trip.

WHAT CHANGED AT EACH STEP?

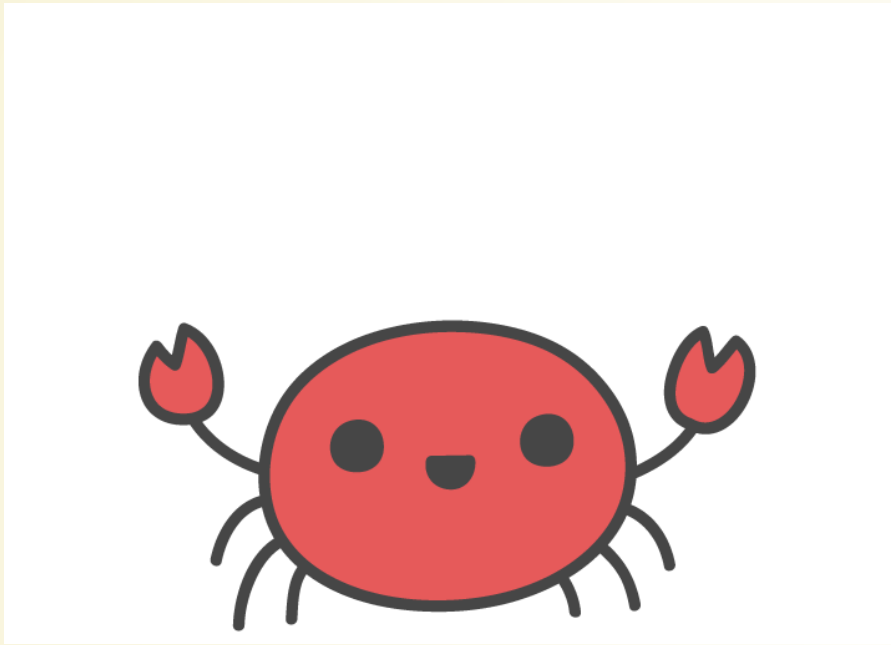
Stage	New boundary	Main lesson
QEMU	register contract	hardware is an interface
01	VFS	character device lifecycle
02	MMIO	mapped I/O and ordering
03	Device Tree	discovery and resources
04–05	ioctl/workqueue	command ABI and deferred work
06	IRQ	hardware-driven completion

REPOSITORY MAP

```
device/qemu/          virtual RGB peripheral
driver/stages/        five reproducible Rust driver stages
driver/*.py           user-space ioctl client
integrations/         minimal QEMU and Buildroot patches
buildroot-external/   kernel configuration and source injection
slides/               this presentation
```

`make check` boots the VM and verifies the complete write/MMIO path.

THANKS!



Cyril Marin · René Ribaud